FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	
Section:	
Professor:	
Date:	

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import random
import agentpy as ap
import matplotlib.pyplot as plt


class RandomWalker(ap.Agent):
    """An agent that takes one random cardinal step per model tick."""

    def setup(self):
        self.position = (0, 0)
        self.path = [self.position]

    def step(self):
        dx, dy = random.choice([(1, 0), (-1, 0), (0, 1), (0, -1)])
        x, y = self.position
        limit = self.model.p.grid_size - 1

        # Keep each agent inside the square grid.
        self.position = (
            max(0, min(limit, x + dx)),
            max(0, min(limit, y + dy)),
        )
        self.path.append(self.position)


class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.num_agents, RandomWalker)

        # Give each agent a random starting cell and record it.
```

```
        for agent in self.agents:
            agent.position = (
                self.random.randint(0, self.p.grid_size - 1),
                self.random.randint(0, self.p.grid_size - 1),
            )
            agent.path = [agent.position]

    def step(self):
        self.agents.step()


def read_positive(prompt, default):
    """Read a positive integer, using the default for blank input."""
    while True:
        raw = input(f"{prompt} [{default}]: ").strip()
        if not raw:
            return default

        try:
            value = int(raw)
            if value > 0:
                return value
        except ValueError:
            pass

        print("Please enter a whole number greater than zero.")


def main():
    print("Random Walk Simulation (AgentPy)")
    num_agents = read_positive("Number of agents", 5)
    grid_size = read_positive("Grid size (N for an N x N grid)", 20)
    num_steps = read_positive("Number of steps", 30)

    parameters = {
        "num_agents": num_agents,
        "grid_size": grid_size,
        "steps": num_steps,
        "seed": 7,
    }

    model = RandomWalkModel(parameters)
    model.run()
```

```
# Plot each agent's route. Squares mark starts; stars mark endpoints.
fig, ax = plt.subplots(figsize=(8, 7))
colors = plt.cm.tab10.colors

for i, agent in enumerate(model.agents):
    xs = [point[0] for point in agent.path]
    ys = [point[1] for point in agent.path]
    color = colors[i % len(colors)]

    ax.plot(
        xs,
        ys,
        marker="o",
        markersize=2,
        linewidth=1.4,
        alpha=0.85,
        color=color,
        label=f"Agent {i + 1}",
    )
    ax.scatter(xs[0], ys[0], marker="s", s=45, color=color, edgecolor="black")
    ax.scatter(xs[-1], ys[-1], marker="*", s=90, color=color, edgecolor="black")

ax.set_xlim(-0.5, grid_size - 0.5)
ax.set_ylim(-0.5, grid_size - 0.5)
ax.set_xticks(range(grid_size))
ax.set_yticks(range(grid_size))
ax.grid(True, alpha=0.25)
ax.set_aspect("equal", adjustable="box")
ax.set_title(f"Bounded random walks: {num_agents} agents, {num_steps} steps")
ax.set_xlabel("Grid column")
ax.set_ylabel("Grid row")
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), fontsize="small")
fig.tight_layout()
plt.show()

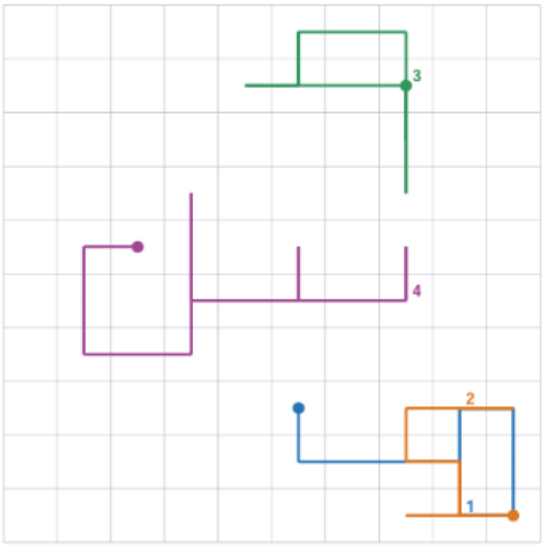
print("\nSimulation complete.")
print(f"Agents: {num_agents} | Grid: {grid_size} x {grid_size} | Steps: {num_steps}")
for i, agent in enumerate(model.agents, start=1):
    print(f"Agent {i}: start={agent.path[0]}, end={agent.path[-1]}")

# Print responses to the assessment discussion questions.
print("\nDiscussion")
print("1. With many agents on a small grid, paths overlap and cells become crowded.")
print("   Agents move independently, so they do not block one another.")
```

```
    print("2. More steps give agents more chances to move away from their start,")
    print("   usually increasing the area their paths cover. The grid limits the spread.")
    print("3. Agents do not leave the grid because each move is clamped to its boundaries.")
    print("4. Random walks can represent diffusion, crowd movement, or animal foraging.")
    print("   Obstacles, preferred directions, and agent interactions can make the model")
    print("   more realistic.")


if __name__ == "__main__":
    main()
```

Sample output



Example: 4 agents, 10 x 10 grid, 18 steps (illustrative seeded run)

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. **With many agents on a small grid, more agents occupy the same cells and their paths overlap, making the chart crowded. Agents move independently, so they don't block one another.**
2. **More steps give agents more chances to move away from their starting points, so their paths usually cover a wider area. The spread is still limited by the grid boundaries.**

3. **No. Each move is clamped to the grid's minimum and maximum coordinates, so an agent that reaches an edge stays within the grid.**
4. **The simulation can represent simple movement patterns: particles spreading through space, people moving through an area, or animals foraging. To model these more realistically, you could add rules for obstacles, preferred directions, agent interactions, or different movement probabilities.**